UNIT – V

FILE

Why files are needed?

- When a program is terminated, the entire data is lost. Storing in a file will preserve your data even if the program terminates.
- If you have to enter a large number of data, it will take a lot of time to enter them all. However, if you have a file containing all the data, you can easily access the contents of the file using few commands in C.
- You can easily move your data from one computer to another without any changes.

File I/O:-

Sometimes it is necessary to store the data in a manner that can be later retrieved and displayed either in a part or in whole. This medium is usually a "file" on the disk. File I/O can be handled by using different functions.

a) Formatted functions:- The file input function fscanf( ) and the file output function fprintf( ) are called formatted file I/O functions.

b)Unformatted functions:- The input functions like getc( ), getw( ), and fread( ) are called unformatted file input functions and putc( ), putw( ), and fwrite( ) functions are unformatted file output functions. Each and every function is having its own syntax and meaning.

File streams:- Stream is either reading or writing of data. The streams are designed to allow the user to access the files efficiently. A stream is a file or physical device like key board, printer, monitor, etc., The FILE object uses these devices. When a C program is started, the operating system is responsible for opening three streams: standard input stream (**stdin**), standard output stream (**stdout**), standard error(**stderr**).Normally the stdin is connected to the keyboard, the stdout and stderr are connected to the monitor.

Files

File is a collection of bytes that is stored on secondary storage devices like Hard disk.

OR

A **file** represents a sequence of bytes on the disk where a group of related data is stored. File is created for permanent storage of data. It is a ready made structure.

Note:

All files related function are available in **stdio.h** header file.

Types of Files

When dealing with files, there are two types of files you should know about:

1. Text files
2. Binary files

1. Text files

Text files are the normal .txt files that you can easily create using Notepad or any simple text editors.

When you open those files, you'll see all the contents within the file as plain text. You can easily edit or delete the contents.

They take minimum effort to maintain, are easily readable, and provide least security and takes bigger storage space.

2. Binary files

Binary files are mostly the .bin files in your computer.

Instead of storing data in plain text, they store it in the binary form (0's and 1's).

They can hold higher amount of data, are not readable easily and provides a better security than text files.

File Operations

In C, you can perform four major operations on the file, either text or binary:

- Naming a file/Creation of new file
- Opening an existing file
- Reading data from file
- Writing data into file
- Closing a file

Steps for processing a file

- Declare a file pointer
- open a file using fopen() function
- Process the file using suitable file functions.
- close the file using fclose() function.

Declaration of a file

When working with files, you need to declare a pointer of type file. This declaration is needed for communication between the file and program.

Syntax

FILE ***fp**;

Opening a file - for creation and edit

The fopen() function is used to create a new file or to open an existing file.

General Syntax :

fp = fopen("fileopen","mode")

For Example:

fopen("E:\\cprogram\\newprogram.txt","w");

fopen("E:\\cprogram\\oldprogram.bin","rb");

Closing a File

The file (both text and binary) should be closed after reading/writing.

Closing a file is performed using library function fclose().

fclose(fptr); //fptr is the file pointer associated with file to be closed.

File Opening Modes

Mode	Description
r	opens a text file in read mode
w	opens a text file in write mode
a	opens a text file in append mode
r+	opens a text file in read and write mode
w+	opens a text file in read and write mode
a+	opens a text file in read and write mode
rb	opens a binary file in read mode

wb	opens a binary file in write mode
ab	opens a binary file in append mode
rb+	opens a binary file in read and write mode
wb+	opens a binary file in read and write mode
ab+	opens a binary file in read and write mode

Difference between Append and Write Mode

Write (w) mode and Append (a) mode, while opening a file are almost the same. Both are used to write in a file. In both the modes, new file is created if it doesn't exists already.

The only difference they have is, when you open a file in the write mode, the file is reset, resulting in deletion of any data already present in the file. While in append mode this will not happen. Append mode is used to append or add data to the existing data of file(if any). Hence, when you open a file in Append(a) mode, the cursor is positioned at the end of the present data in the file.

Formatted File I/O Functions

Syntax of fprintf is

fprintf (fp, “control string”, list);

Example: fprintf(fp1, “%s %d”, name, age);

Syntax of fscanf is,

fscanf(fp, “control string”, list);

Example: fscanf(fp, “%s %d”, name, & age);

Note:

- fscanf is used to read list of items from a file
- fprintf is used to write a list of items to a file.

Note:

EOF – End of file (when EOF encountered the reading / writing should be terminated)

Example:

```
#include <stdio.h>
  main(){
    FILE *fp;
    fp = fopen("file.txt", "w");//opening file
  fprintf(fp, "Hello file by fprintf...\n");//writing data into file
    fclose(fp);//closing file
  }
```

Example 1: Write to a text file using fprintf()

```
#include <stdio.h>
int main()
{
  int num;
  FILE *fptr;
  fptr = fopen("C:\\program.txt","w");
  if(fptr == NULL)
  {
    printf("Error!");
    exit(1);
  }
  printf("Enter num: ");
  scanf("%d",&num);
  fprintf(fptr,"%d",num);
  fclose(fptr);
  return 0;
}
```

Example 2: Read from a text file using fscanf()

```
#include <stdio.h>
int main()
{
  int num;
  FILE *fptr;
  if ((fptr = fopen("C:\\program.txt","r")) == NULL){
      printf("Error! opening file");
      // Program exits if the file pointer returns NULL.
      exit(1);
  }
  fscanf(fptr,"%d", &num);
  printf("Value of n=%d", num);
  fclose(fptr);
    return 0;
}
```

Input/Output Operation on files

To perform Input/Output Operation on files we need below functions.

S.No	Function	Operation	Syntax
1	getc()	Read a character from a file	getc(fp)
2	putc()	Write a character in file	putc(c, fp)

3	fprintf()	To write set of data in file	fprintf(fp, "control string", list)
4	fscanf()	To read set of data from file.	fscanf(fp, "control string", list)
5	getw()	To read an integer from a file.	getw(fp)
6	putw()	To write an integer in file.	putw(integer, fp)

Unformatted File I/O Functions

fputc() function

The fputc() function is used to write a single character into file.

putc ( ):-Putting a character in to the file. It works with only character data type. One character at a time can write into a file.

Ex: char ch =’a’;

putc (ch, fp);

Example:

```
#include <stdio.h>
main(){
  FILE *fp;
  fp = fopen("file1.txt", "w");//opening file
  fputc('a',fp);//writing single character into file
  fclose(fp);//closing file
}
```

file1.txt

```
a
```

fgetc() function

The fgetc() function returns/read a single character from the file. It gets a character from the stream. It returns EOF at the end of file.

getc ( ): getting a character from the file, or reading the file information character by character at a time, upto the end of the file by using this function.

Ex:

```
char ch;
ch = getc (fp);
```

Example:

```
#include<stdio.h>
#include<conio.h>
void main(){
FILE *fp;
char c;
clrscr();
fp=fopen("myfile.txt","r");
 while((c=fgetc(fp))!=EOF){
printf("%c",c);
}
fclose(fp);
getch();
}
```

myfile.txt

this is simple text message

fputs()

The fputs() function writes a line of characters into file

Example:

```
#include<stdio.h>
```

```
#include<conio.h>
void main(){
FILE *fp;
clrscr();
fp=fopen("myfile2.txt","w");
fputs("hello c programming",fp);
  fclose(fp);
getch();
}
```

myfile2.txt

hello c programming

fgets()

The fgets() function reads a line of characters from file.

Example:

```
#include<stdio.h>
#include<conio.h>
void main(){
FILE *fp;
char text[300];
clrscr();
  fp=fopen("myfile2.txt","r");
printf("%s",fgets(text,200,fp));

fclose(fp);
```

```
getch();
}
```

Output:

hello c programming

The getw and putw functions:

These are integer oriented functions. These are similar to above functions and are used to read and write integer values. These are useful when we deal with only integer data. The general format is

putw ( ): putting or writing_of an integer_value to a file.

putw (integer , fp);

Ex:

```
int  x = 5;
putw(x,fp);
```

getw ( ): getting or reading integer value from a file.

Ex:

```
int x;
x = getw (fp);
```

File Positioning Functions

fseek()

The fseek() function is used to set the file pointer to the specified offset. It is used to write data into file at desired location.

syntax:

fseek(FILE * stream, long int offset, int whence)

The first parameter stream is the pointer to the file. The second parameter is the position of the record to be found, and the third parameter specifies the location where the offset starts.

Different Whence in fseek	
Whence	Meaning
SEKK_SET	Starts the offset from the beginning of the file.
SEKK_END	Starts the offset from the end of the file.
SEKK_CUR	Starts the offset from the current location of the cursor in the file.

or

fseek(file pointer, offset, position);

- *file pointer* is a pointer to the concerned file.
- *Offset* is a number or variable of type long, it specifies the number of positions (bytes) to be moved from the location specified. If offset is positive number, then moving forward or negative meaning move backwards.
- *Position* is a n integer number and it specifies from which position the file pointer to be moved. Position can take one of the following three values.

  0 beginning of file
  1 current position
  2 end of file

Eg: fseek (fp, 0L,0); - go to the beginning of the file. (Similar to rewind).
fseek (fp, 0L,1); - Stay at current position (Rarely used)
fseek (fp, 0L,2); -go to the end of the file, past the last character of the file.

Example:

```
#include <stdio.h>

 void main(){
```

```
    FILE *fp;
      fp = fopen("myfile.txt","w+");
    fputs("This is javatpoint", fp);
       fseek( fp, 7, SEEK_SET );
    fputs("sonoo jaiswal", fp);
    fclose(fp);
  }
```

myfile.txt

This is sonoo jaiswal

rewind()

This function places the file pointer to the beginning of the file, irrespective of where it is present right now. It takes file pointer as an argument.

Syntax:

rewind(fp);

Example:

File: file.txt

```
this is a simple text
```

Example:

```
#include<stdio.h>
#include<conio.h>
void main(){
```

```
FILE *fp;
char c;
clrscr();
fp=fopen("file.txt","r");
while((c=fgetc(fp))!=EOF){
printf("%c",c);
}
  rewind(fp);//moves the file pointer at beginning of the file
  while((c=fgetc(fp))!=EOF){
printf("%c",c);
}
  fclose(fp);
getch();
}
```

Output:

this is a simple textthis is a simple text

As you can see, rewind() function moves the file pointer at beginning of the file that is why "this is simple text" is printed 2 times. If you don't call rewind() function, "this is simple text" will be printed only once.

ftell()

The ftell() function returns the current file position of the specified stream. We can use ftell() function to get the total size of a file after moving file pointer at the end of file. We can use SEEK_END constant to move the file pointer at the end of file.

syntax:

n = ftell(fp);

n would give the relative offset(in bytes).

Example:

```
#include <stdio.h>
#include <conio.h>
void main (){
  FILE *fp;
  int length;
  clrscr();
  fp = fopen("file.txt", "r");
  fseek(fp, 0, SEEK_END);
    length = ftell(fp);
    fclose(fp);
  printf("Size of file: %d bytes", length);
  getch();
}
```

Output:

Size of file: 21 bytes

INBUILT FUNCTIONS FOR FILE HANDLING IN C LANGUAGE:

File handling functions	**Description**
fopen ()	fopen () function creates a new file or opens an existing file.
fclose ()	fclose () function closes an opened file.
getw ()	getw () function reads an integer from file.

putw ()	putw () functions writes an integer to file.
fgetc ()	fgetc () function reads a character from file.
fputc ()	fputc () functions write a character to file.
gets ()	gets () function reads line from keyboard.
puts ()	puts () function writes line to o/p screen.
fgets ()	fgets () function reads string from a file, one line at a time.
fputs ()	fputs () function writes string to a file.
feof ()	feof () function finds end of file.
fgetchar ()	fgetchar () function reads a character from keyboard.
fprintf ()	fprintf () function writes formatted data to a file.
fscanf ()	fscanf () function reads formatted data from a file.
fputchar ()	fputchar () function writes a character onto the output screen from keyboard input.
fseek ()	fseek () function moves file pointer position to given location.
SEEK_SET	SEEK_SET moves file pointer position to the beginning of the file.

SEEK_CUR	SEEK_CUR moves file pointer position to given location.
SEEK_END	SEEK_END moves file pointer position to the end of file.
ftell ()	ftell () function gives current position of file pointer.
rewind ()	rewind () function moves file pointer position to the beginning of the file.
getc ()	getc () function reads character from file.
getch ()	getch () function reads character from keyboard.
getche ()	getche () function reads character from keyboard and echoes to o/p screen.
getchar ()	getchar () function reads character from keyboard.
putc ()	putc () function writes a character to file.
putchar ()	putchar () function writes a character to screen.
printf ()	printf () function writes formatted data to screen.
sprinf ()	sprinf () function writes formatted output to string.
scanf ()	scanf () function reads formatted data from

	keyboard.
sscanf ()	sscanf () function Reads formatted input from a string.
remove ()	remove () function deletes a file.
fflush ()	fflush () function flushes a file.